

Lecture 1 - Basics: methods, assertions, functions, pre- and post-conditions

Functional verification is the task of proving properties of a program's input-output behavior. In this course, we shall use Dafny, a programming language that combines a specification language and a proof assistant for verifying programs. In this lecture, Dafny's basics are illustrated via simple examples. The concepts of assertion, pre- and post-conditions are introduced, together with the notions of proof obligation and control paths.

Methods and assertions

Dafny is an object-oriented programming language with a functional extension. Although **methods** are defined in classes, they can also be stand-alone procedures. Dafny is a strongly typed language (no casting is allowed); variables and expressions are typed. The body of a method may contain several expressions and statements, including variable declarations:

```
var <varname> : <type>;
```

or commands like assignments:

```
[var]<varname> := <expression>;
```

Other commands are conditionals or case selections, iterations (just while) and method calls.

Commands are state transformers, where the **state** is a mapping from variables to values. **Assertions** are predicates of the state. A **predicate** is a boolean-valued expression that may contain logical connectives and quantifiers.

The **assert** statement:

```
assert <predicate>;
```

does not have the effect of blocking the execution and raising an exception in case of failure (as for similar statements, e.g., in the assert library of C). It is a proof obligation: Dafny is asked to prove that it must be true whenever the control reaches that assert statement.

```

method SquareDiff(x: real, y: real) returns (z: real)
{
    var u : real := (x + y) * (x - y); // local variable
    // the type real for u can be omitted as it is deduced
    // from the type of (x + y) * (x - y)
    z := u;
    // redundant writing: it suffices
    // z := (x + y) * (x - y);
    // or equivalently
    // return (x + y) * (x - y);
    assert z == x * x - y * y;
    // this has to be proved to hold for any possible execution
    // of the method when it reaches this point
}

```

More on assertions and execution paths

A sequence of program points that can be reached in some execution is called an **execution path** or just a path. Each command in the path may affect the state. An assertion is checked to hold for any state obtained from an initial one; we shall call the **pre-state** of the method after the modifications induced by the previous commands in the path that leads to it.

Let's consider the following example:

```

method Triple(x: int) returns (r: int) {
    var y := 2 * x;
    // x := 0; error: input var. x is immutable
    r := x + y;
}

```

We can use assertions to ask Dafny to verify whether the predicate defined by a logical formula should be true of any state that can be reached at a certain point in the execution of the method (procedure):

```

method Triple1(x: int) returns (r: int) {
    var y := 2 * x;
    r := x + y;
    assert r == 3 * x; // proof obligation
}

```

Assertions are considered along the execution path, leading to the point where the assert command is issued. Suppose a false assertion is on the path. In that case, all the following assertions on the path are unreachable and treated as trivially true, since in logic, false implies anything (*law ex-falso-quodlibet*). This is shown in the second version of the Triple method

```

method Triple2(x: int) returns (r: int) {
    var y := 2 * x;
    assert false;
    r := x + y;
    assert r == 3 * x + 1; // it holds since this point is unreachable
}

```

and in the third version of it below:

```
method Triple3(x: int) returns (r: int) {
  var y := 2 * x;
  r := x + y;
  assert r == 10 * x; // error: does not hold in general, but only if x == 0
  assert r < 5; // it holds because this point is reached when x == 0
  assert false; // error: cannot hold ever
}
```

Pre and post-conditions: contracts

Pre and post-conditions are assertions associated with a method using the keywords **requires** and **ensures** respectively.

The **pre-condition** is a hypothesis on the pre-state, namely on the values of the method parameters. If the pre-condition fails of the pre-state, then the method verifies trivially. Otherwise, the method is expected to terminate with a post-state that satisfies the **post-condition**.

The pair of pre- and post-conditions is called a **contract**. The metaphor is that the pre-condition formalizes the rights of the method, while the post-condition is its duties.

```
method Triple4(x: int) returns (r: int)
  requires x % 2 == 0
  ensures  r == 3 * x
{
  var y := x / 2; // integer division is rounded
  r := 6 * y;
}
```

If the pre-condition is empty, namely trivially true, it can be omitted.

```
method SquareDiff'(x: real, y: real) returns (z: real)
  ensures z == x * x - y * y
{
  z := (x + y) * (x - y);
}
```

Functions

Functions consist of just one expression and are pure, namely, they cannot affect the state. They can be used to both calculate values and specify method behaviour in assertions.

Functions can be used in the specifications as mathematical definitions, and can be specified themselves by requires-ensures clauses.

```

function Abs(x: int): int
  ensures Abs(x) >= 0
  ensures Abs(x) == x || Abs(x) == - x
  // multiple ensures clauses are interpreted as the
  // conjunction of the respective predicates
  // (similarly for multiple requires)
{
  if x < 0 then - x else x
}

```

The next method is what in C is called a "function", and it is reported here to show how the function Abs can be used in the contract of a method computing the absolute value of an integer.

```

method AbsMet(x: int) returns (y: int)
  ensures y == Abs(x)
{
  if x < 0 {y := - x; }
  else { y := x; }
  // differently than in case of expressions,
  // the command if-then-else
  // requires using blocks {...}
}

```

Using assertions, methods can be tested without actually running.

```

method TestAbs()
{
  var r := - 123;
  ghost var p := r;
  r := AbsMet(r);
  assert r == - p;
}

```

This is the key fact in functional verification with a tool such as Dafny, namely programs are just statically checked by reasoning on the code and on the specification clauses it contains, not by executing the program whatsoever.

Reference: Leino chapter 1.